

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CNTT&TT



BTVN NGÀY 5/5
MÔN HỌC: LẬP TRÌNH MẠNG

Giáo viên hướng dẫn: Lê Bá Vui

Sinh viên thực hiện: Trần Anh Phú

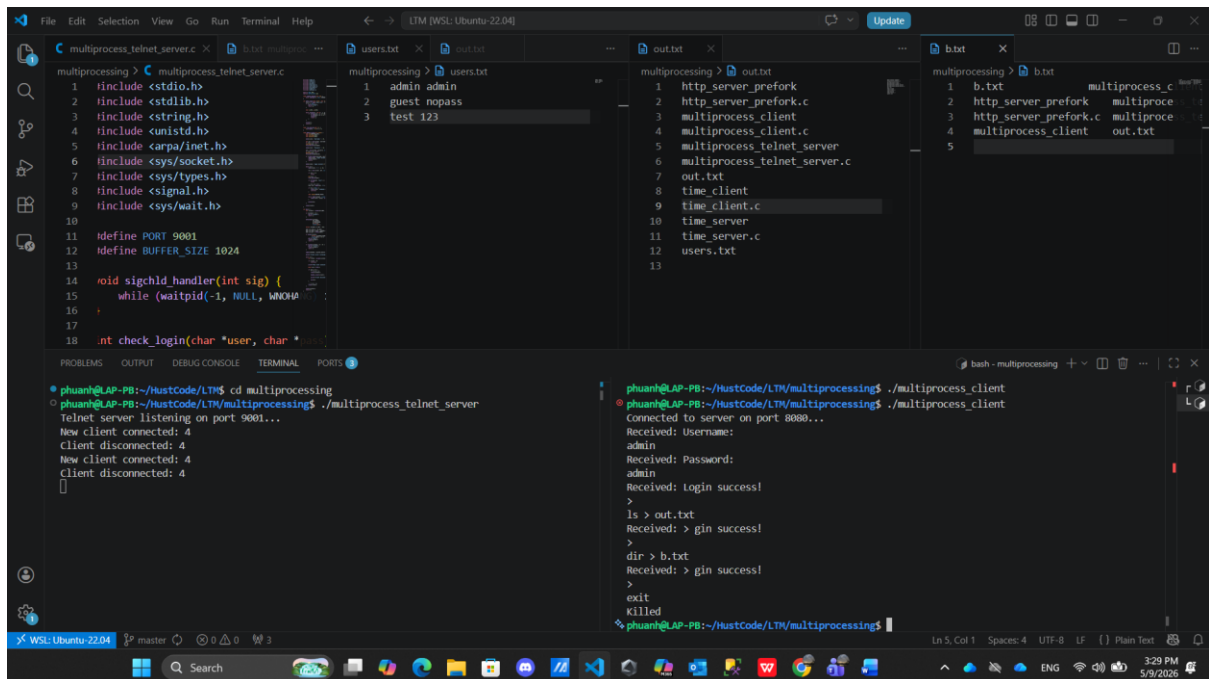
Mã sinh viên: 20235397

Lớp: Lập trình mạng – 168503

Kỳ học 2025.2

1) Lập trình lại ứng dụng telnet_server (slide 174) sử dụng kỹ thuật multiprocessing

Kết quả:



```
File Edit Selection View Go Run Terminal Help
multiprocessing > C multiprocess_telnet_server.c
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <unistd.h>
5 #include <arpa/inet.h>
6 #include <sys/socket.h>
7 #include <sys/types.h>
8 #include <signal.h>
9 #include <sys/wait.h>
10
11 #define PORT 9001
12 #define BUFFER_SIZE 1024
13
14 void sighandler(int sig) {
15     while (waitpid(-1, NULL, WNOHUP) > 0) {}
16 }
17
18 int check_login(char *user, char *pass) {
19     FILE *fp = fopen("users.txt", "r");
20     if (fp == NULL) {
21         printf("Error opening users.txt\n");
22         return 0;
23     }
24     char line[256];
25     while (fgets(line, sizeof(line), fp) != NULL) {
26         char *token = strtok(line, " ");
27         if (token == NULL) continue;
28         if (strcmp(token, user) == 0) {
29             token = strtok(NULL, " ");
30             if (token != NULL) {
31                 if (strcmp(token, pass) == 0) {
32                     return 1;
33                 }
34             }
35         }
36     }
37     return 0;
38 }
39
40 int main() {
41     int sock = socket(AF_INET, SOCK_STREAM, 0);
42     if (sock < 0) {
43         perror("socket");
44         return 1;
45     }
46     struct sockaddr_in server_addr;
47     server_addr.sin_family = AF_INET;
48     server_addr.sin_port = htons(PORT);
49     inet_pton(AF_INET, "0.0.0.0", server_addr.sin_addr.s_addr);
50     if (bind(sock, (struct sockaddr *)&server_addr, sizeof(server_addr)) < 0) {
51         perror("bind");
52         return 1;
53     }
54     if (listen(sock, 5) < 0) {
55         perror("listen");
56         return 1;
57     }
58     printf("Telnet server listening on port 9001...\n");
59     while (1) {
60         struct sockaddr_in client_addr;
61         socklen_t len = sizeof(client_addr);
62         int conn = accept(sock, (struct sockaddr *)&client_addr, &len);
63         if (conn < 0) {
64             perror("accept");
65             continue;
66         }
67         printf("New client connected: %s\n", inet_ntoa(client_addr.sin_addr));
68         pid_t pid = fork();
69         if (pid < 0) {
70             perror("fork");
71             close(conn);
72             continue;
73         }
74         if (pid == 0) {
75             // Child process
76             close(sock);
77             char buf[BUFFER_SIZE];
78             while (1) {
79                 int n = read(conn, buf, sizeof(buf));
80                 if (n < 0) {
81                     perror("read");
82                     break;
83                 }
84                 if (n == 0) {
85                     printf("Client disconnected: %s\n", inet_ntoa(client_addr.sin_addr));
86                     break;
87                 }
88                 buf[n] = '\0';
89                 if (strcmp(buf, "exit") == 0) {
90                     printf("Killed\n");
91                     break;
92                 }
93                 if (strcmp(buf, "login") == 0) {
94                     char user[256], pass[256];
95                     printf("Received: Username: ");
96                     fgets(user, sizeof(user), stdin);
97                     printf("Received: Password: ");
98                     fgets(pass, sizeof(pass), stdin);
99                     if (check_login(user, pass)) {
100                         printf("Received: Login success!\n");
101                     } else {
102                         printf("Received: Login failed!\n");
103                     }
104                 } else if (strcmp(buf, "ls") == 0) {
105                     system("ls > out.txt");
106                     printf("Received: > ls success!\n");
107                 } else if (strcmp(buf, "dir") == 0) {
108                     system("dir > b.txt");
109                     printf("Received: > dir success!\n");
110                 } else {
111                     printf("Received: %s\n", buf);
112                 }
113             }
114             close(conn);
115         } else {
116             // Parent process
117             close(conn);
118         }
119     }
120     return 0;
121 }

multiprocessing > users.txt
1 admin admin
2 guest nopass
3 test 123

multiprocessing > out.txt
1 http_server_prefork
2 http_server_prefork.c
3 multiprocessing_client
4 multiprocessing_client.c
5 multiprocessing_telnet_server
6 multiprocessing_telnet_server.c
7 out.txt
8 time_client
9 time_client.c
10 time_server
11 time_server.c
12 users.txt
13

multiprocessing > b.txt
1 b.txt multiprocessing_c
2 http_server_prefork multiproc
3 http_server_prefork.c multiproc
4 multiprocessing_client out.txt
5

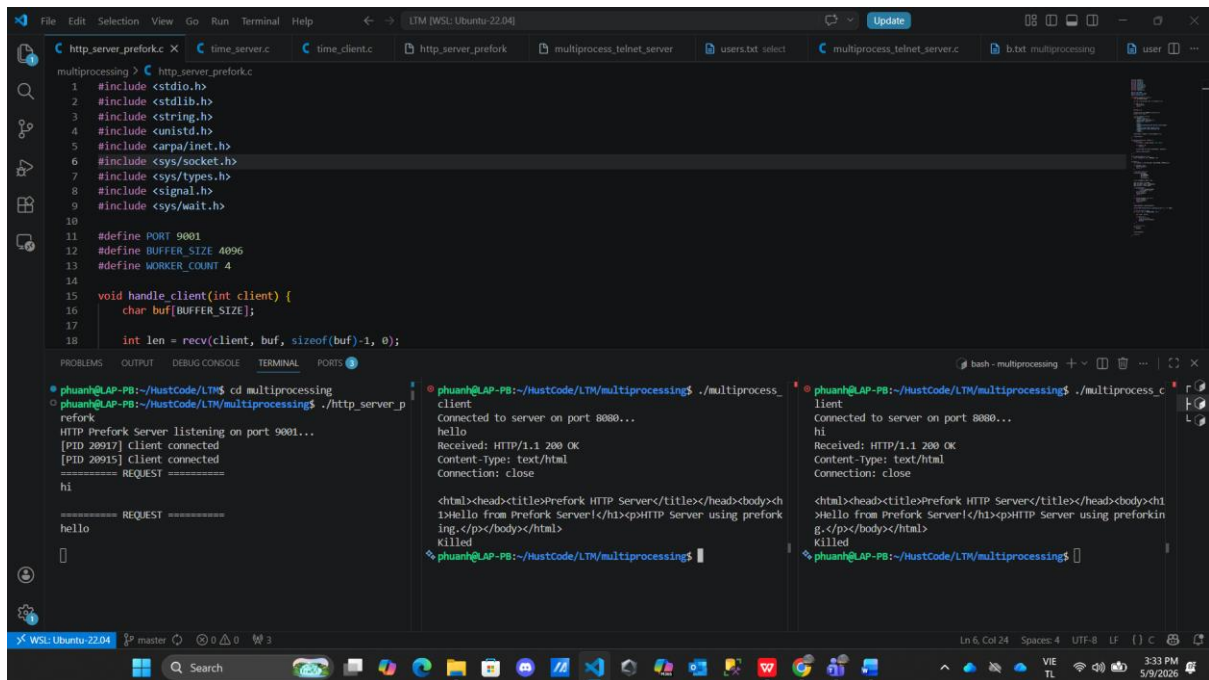
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
phuanh@LAP-PB:~/HustCode/LTM/multiprocessing$ cd multiprocessing
phuanh@LAP-PB:~/HustCode/LTM/multiprocessing$ ./multiprocess_telnet_server
Telnet server listening on port 9001...
New client connected: 4
Client disconnected: 4
New client connected: 4
Client disconnected: 4

phuanh@LAP-PB:~/HustCode/LTM/multiprocessing$ ./multiprocess_client
Connected to server on port 8080...
Received: Username:
admin
Received: Password:
admin
Received: Login success!
>
ls > out.txt
Received: > ls success!
>
dir > b.txt
Received: > dir success!
>
exit
Killed
phuanh@LAP-PB:~/HustCode/LTM/multiprocessing$
```

Server được chạy và mở kết nối ở cổng 9001. Client thực hiện kết nối đến Server. Client phải nhập đúng Tài khoản để đăng nhập nếu không sẽ bị killed. Khi Client đã kết nối thành công đến Server, Client thực hiện lệnh theo cú pháp tương ứng. Nội dung của lệnh sẽ được ghi ra file tương ứng theo cú pháp.

2. Lập trình lại ứng dụng http_server (slide 154) sử dụng kỹ thuật preforking

Kết quả:

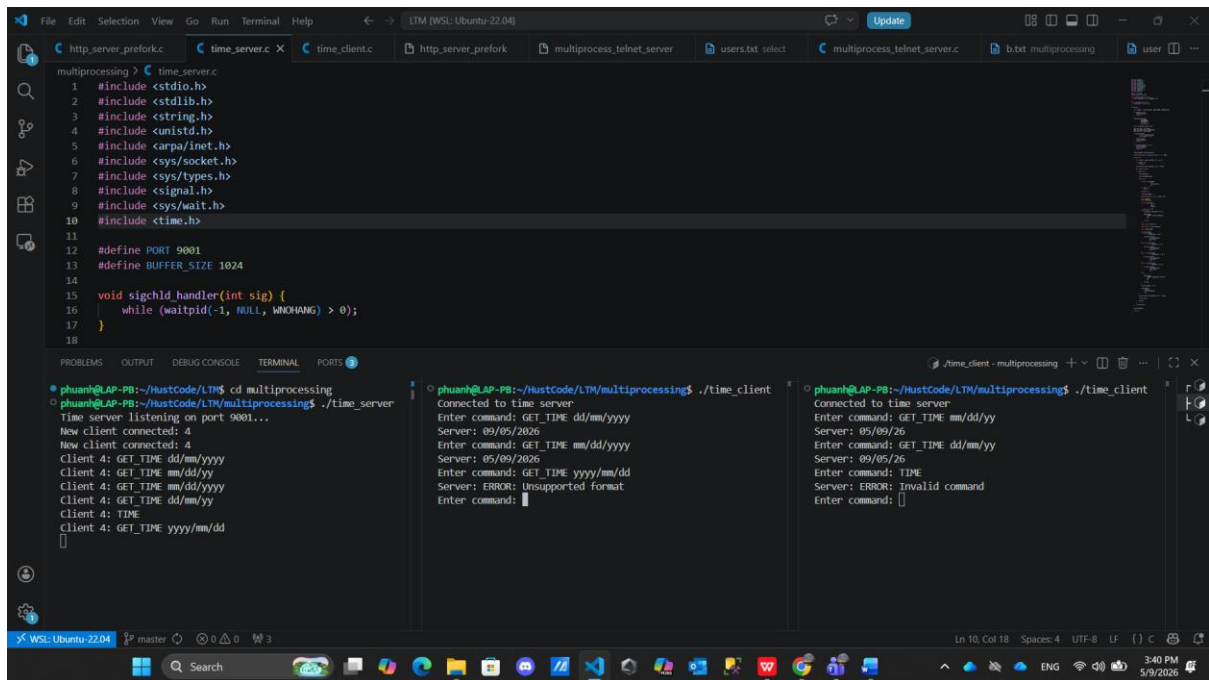


Server được chạy và mở kết nối ở cổng 9001. Client thực hiện kết nối đến Server. Client thực hiện gửi request bất kì đến Server. Server sẽ gửi response lại cho mỗi Client.

3. Lập trình ứng dụng time_server thực hiện chức năng sau:

- Chấp nhận nhiều kết nối từ các client sử dụng kỹ thuật multiprocessing.
- Client gửi lệnh GET_TIME [format] để nhận thời gian từ server.
- format là định dạng thời gian server cần trả về client. Các format cần hỗ trợ gồm:
 - dd/mm/yyyy – vd: 30/01/2023
 - dd/mm/yy – vd: 30/01/23
 - mm/dd/yyyy – vd: 01/30/2023
 - mm/dd/yy – vd: 01/30/23
- Cần kiểm tra tính đúng đắn của lệnh client gửi lên Server

Kết quả:



```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <unistd.h>
5 #include <arpa/inet.h>
6 #include <sys/socket.h>
7 #include <sys/types.h>
8 #include <signal.h>
9 #include <sys/wait.h>
10 #include <time.h>
11
12 #define PORT 9001
13 #define BUFFER_SIZE 1024
14
15 void sigchld_handler(int sig) {
16     while (waitpid(-1, NULL, WNOHANG) > 0);
17 }
18
```

```
phuanh@LAP-PB:~/HustCode/LTM$ cd multiprocessing
phuanh@LAP-PB:~/HustCode/LTM/multiprocessing$ ./time_server
Time server listening on port 9001...
New client connected: 4
Client 4: GET_TIME dd/mm/yyyy
Server: 09/05/2026
Client 4: GET_TIME mm/dd/yy
Server: 09/09/26
Client 4: GET_TIME mm/dd/yyyy
Server: 09/09/26
Client 4: GET_TIME dd/mm/yy
Server: 09/05/26
Client 4: TIME
Server: ERROR: Invalid command
Client 4: GET_TIME yyyy/mm/dd
Server: ERROR: Unsupported format
Enter command:

```

Server được chạy và mở kết nối ở cổng 9001. Client thực hiện kết nối đến Server. Client thực hiện gửi lệnh và nhận phản hồi tương ứng từ Server.